keyval

A minimal key-value format for flat pairs and implicit trees via dotted paths.

File extension: `.kv`

Syntax

Files are UTF-8. Every line ends with LF, including the last. The file may be empty.

The first line may begin with `#`; its content is unspecified. No other line may begin with `#`.

Empty lines are not permitted. No line may end in a space. A line is either a value pair or a marker:

```
key value
key
```

In a value pair, the key ends at the first space and the value is the rest of the line. The value may itself begin with spaces.

In a marker, the key stands alone with no space. It declares an empty node at that path.

Keys

A key is a dotted path of one or more segments:

```
segment[.segment]*
```

A segment is either a name or an index. The first character decides: a letter starts a name, a digit starts an index.

A name matches `[A-Za-z][A-Za-z0-9_]*`: a letter followed by letters, digits or underscores. Case counts: `Web` and `web` are different names. Uppercase letters are permitted from version 1.1.

An index matches `[0-9]+`: one or more digits. Its value is the denoted integer. Leading zeros are padding.

The alphabet is chosen for the sort. Every character a segment may hold, the letters and the digits and the underscore, sorts above the dot that separates segments. That is what keeps a subtree contiguous and puts a node's own lines ahead of any sibling whose name extends it. A hyphen sorts below the dot, so `web-small.cores` would come before `web.cores` and a file sorted by eye would be out of order to the parser; write a hyphenated name, a hostname above all, with underscores. Uppercase passes the same test, and its cost is cosmetic: it sorts below both the underscore and the lowercase letters, so `Web` and `web` do not sit together.

Tree structure

The dotted path defines a tree implicitly. A node holds children keyed by name or by index, and both kinds may appear under the

same node. Named children form an associative array. Indexed children form an ordered sequence.

A node may not be both a leaf and a branch. A leaf carries a value; a branch has children. Mixing them is a parse error.

Markers

A marker asserts that a node exists at the given path and is currently empty. A marker alongside children under the same path is a parse error. An append may leave a stale marker behind, just as it leaves lines out of order; a normalizing pass repairs both. An appended file does not parse until normalized.

A marker whose last segment is an index counts toward the parent's sequence like any other indexed child.

A marker declares an empty node, not an empty value. An empty value is a value pair written with `\0`.

Sequences

The indexed children of a node must have consecutive values starting from 0. Leaves, branches and markers all count.

Within a sequence, every index is written with the same width. Writing `x.0` and `x.01` in the same sequence is a parse error. The width may exceed what the count requires; a short sequence written `00`, `01` can grow to 100 items without rewriting earlier lines.

Ordering

Files must be sorted in strict lexicographic order on the full key string, compared byte by byte. Locale collation is not used, and the difference is not academic: a locale that skips punctuation on its first pass calls an out-of-order file sorted. `LC_ALL=C sort` is the ordering this section describes. Out-of-order lines are a parse error. Strictness also forbids duplicate keys.

The sort is not numeric-aware. Uniform index width is what keeps sequences in order: equal-width index strings sort numerically. A sequence that outgrows its width must be rewritten one digit wider, `0` through `9` becoming `00` through `10` at the 11th item.

Digits sort before letters, so indexed children precede named children under the same node. The dot sorts before every segment character, so a subtree always forms one contiguous block of lines.

Values

Backslash escape sequences:

Sequence	Meaning
`\\`	Literal backslash
`\n`	Newline
`\t`	Horizontal tab
`\r`	Carriage return
`\0`	End-of-value marker; lets a value end in spaces

A backslash followed by any other character is a parse error.

A value must not contain raw control characters. Newline, tab and carriage return are written as escapes; other control characters cannot be represented.

`\0` may appear only at the end of a value; elsewhere it is a parse error. Since no line may end in a space, `\0` is how empty values and trailing spaces are written: `key \0` holds the empty value, and `key two words \0` holds `two words` followed by one space. Meaning beyond delimiting the value is left to tools and derived formats.

Example

```
build.00 ./configure --prefix=/usr
build.01 make
build.parallel yes
person.0.email \0
person.0.name Charles Ingvar Jönsson
person.1.name Anna
planets
```

`person.0.email` holds an empty value; `planets` declares an empty node. `build` pads its indices to leave room for growth, `person` does not; width is a per-sequence choice.